

ROS2 Spring Workshop

Lab 7: GPS Autosteer (P-Control & Headland Turns)

Project Context: The “Redemption Arc”

Recall Lab 3 (The Invisible Wall): You programmed a “Blind U-Turn” (State 1). You told the robot to turn left for a specific time.

The Failure: The robot never exited the turn straight. It always drifted into the neighbouring row because it was guessing.

The Fix (Lab 7): Today, we stop guessing. We will use **Path Planning** to calculate the perfect U-Turn geometry so the robot enters Row 2 exactly on the centre line.

Objective

We will solve the **Cross-Track Error (CTE)** problem, the lateral drift from the guidance line, using **Path Interpolation** and a **P-Controller**.

You will learn:

- **Linear Interpolation (LERP):** How to slice a long straight line into tiny “Breadcrumbs.”
- **P-Control:** The math behind smooth steering.
- **Headland Turns:** Using geometry to generate a U-Turn path for a tractor.

A Note on Terminology: P vs. PID

You may have heard the term **PID** (Proportional-Integral-Derivative) controller thrown around. Today we are only using the **P** (Proportional) term, one of the three. The I and D terms add features like eliminating steady-state drift and damping oscillation, but they also add complexity and tuning difficulty.

For our path-following needs, pure P-control is sufficient. Full PID is beyond this lab’s scope; ask during office hours if you want to go deeper.

Pre-Lab Concept Primer

1. The CTE Problem

Imagine you are at $(0, 0)$ and want to go to $(6, 3)$.

- **Naive Method:** Calculate the angle to $(6, 3)$ once and drive. If you hit a bump, the robot drifts. It will slowly curve back towards the goal, creating a huge arc (leaving unplanted crops!).
- **The Solution (Breadcrumbs):** Instead of one distant goal nearly 7m away, we create roughly 13 goals spaced 0.5m apart. The robot chases the “Carrot” immediately in front of it, forcing a straight line.

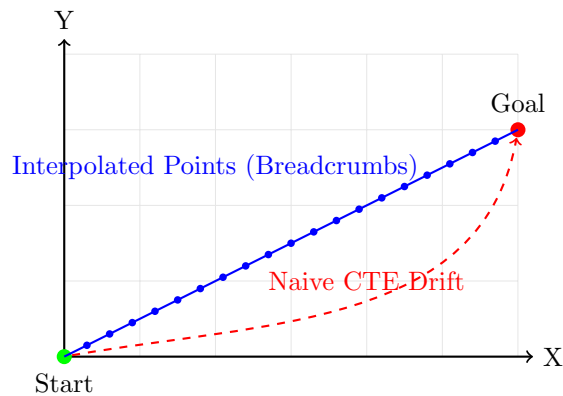


Figure 1: Why we need Breadcrumbs: Simulating infinite points creates a straight line.

2. The P-Controller (The Steering Wheel)

How does the robot decide how *hard* to turn? We use Proportional Control.

$$\text{Steering} = K_p \times \text{Error}$$

- **Error:** The angle difference between where you are facing vs. the goal.
- **Kp (Gain):** The “Aggressiveness” knob.
 - **High Kp:** Robot turns violently (Jittery).
 - **Low Kp:** Robot turns lazily (Drifts off path).
- **Result:** If the error is big, turn hard. If the error is small, turn gently.

3. The “Angle Wrap” Trap (Critical Math)

Robots use Radians ($-\pi$ to $+\pi$).

- If you are at 359° (6.26 rad) and want to go to 1° (0.01 rad), simple math says: $0.01 - 6.26 = -6.25$.
- The robot thinks this is a huge error and spins 360 degrees the wrong way!
- **The Fix:** We must normalize the error so the robot takes the shortest path.

Part 1: Package Setup

We need to create the container for your new navigation brain.

1. Navigate to source:

```
1 cd /workspaces/agtech_ros2/src
2
```

2. Create the package:

```
1 ros2 pkg create --build-type ament_python lab7_autosteer
2
```

Part 2: Importing the Logic

Writing a path-following controller from scratch is tedious. We have provided the core “Straight Line” logic in the course resources. Your job is to install it, analyze it, and then upgrade it to handle turns.

1. Copy the script from resources:

```
1 cp /workspaces/agtech_ros2/lab_resources/lab7/autosteer.py \
2 /workspaces/agtech_ros2/src/lab7_autosteer/lab7_autosteer/
3
```

2. Open the file for analysis:

```
1 code /workspaces/agtech_ros2/src/lab7_autosteer/lab7_autosteer/autosteer.py
2
```

Code Analysis

Read the code below. Pay attention to how `generate_field_path` constructs the path piece by piece.

```
1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from geometry_msgs.msg import Twist
5 from turtlesim.msg import Pose
6 import math
7
8 class Autosteer(Node):
9     def __init__(self):
10         super().__init__('autosteer_node')
11
12         self.pub_cmd = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
13         self.sub_pose = self.create_subscription(Pose, '/turtle1/pose', self.update_pose
14         , 10)
15         self.pose = None
16
17         # PATH PLANNING
18         self.resolution = 0.1
19         self.waypoints = self.generate_field_path()
20         self.current_wp_index = 0
21
22         # P-CONTROLLER GAINS (Tuning)
23         self.Kp_linear = 2.0
24         self.Kp_angular = 4.0
25
26         self.timer = self.create_timer(0.05, self.control_loop)
27         self.get_logger().info(f"Plan Generated. {len(self.waypoints)} points created.")
28
29     def generate_field_path(self):
30         """ Builds the path segment-by-segment. """
31         path = []
32
33         # SEGMENT 1: DRIVE EAST (Row 1)
34         start_x, start_y = 1.0, 1.0
35         end_x, end_y = 9.0, 1.0
36
37         dist = 8.0
38         num_steps = int(dist / self.resolution)
39
40         for i in range(num_steps):
41             alpha = i / num_steps
42             x = start_x + (end_x - start_x) * alpha
43             y = start_y + (end_y - start_y) * alpha
44             path.append((x, y))
45
46         # SEGMENT 2: THE J-TURN (First 90 degrees)
47         center_x = 9.0
48         center_y = 1.5
49         radius = 0.5
50         num_turn_steps = 30
51
52         for i in range(num_turn_steps):
53             # From -90 deg (-1.57) to 0 deg
54             angle = -1.57 + (1.57 * (i / num_turn_steps))
55             x = center_x + radius * math.cos(angle)
56             y = center_y + radius * math.sin(angle)
57             path.append((x, y))
```

```

57
58     # STUDENT CHALLENGE ZONE
59     # TODO 1: Add Segment 3 (Finish the U-Turn)
60     # TODO 2: Add Segment 4 (Drive West / Return to Start)
61
62     return path
63
64 def update_pose(self, msg): self.pose = msg
65
66 def control_loop(self):
67     if self.pose is None: return
68
69     if self.current_wp_index >= len(self.waypoints):
70         self.stop_robot(); return
71
72     target_x, target_y = self.waypoints[self.current_wp_index]
73
74     # 1. Calculate Errors
75     dx = target_x - self.pose.x
76     dy = target_y - self.pose.y
77     distance = math.sqrt(dx**2 + dy**2)
78     angle_to_goal = math.atan2(dy, dx)
79     angle_error = angle_to_goal - self.pose.theta
80
81     # 2. Normalize Angle (Fix the 360 wrap)
82     # This handles the common case where the error is just outside
83     # [-pi, pi]. A more robust form is:
84     #     angle_error = math.atan2(math.sin(angle_error),
85     #                             math.cos(angle_error))
86     if angle_error > math.pi: angle_error -= 2 * math.pi
87     elif angle_error < -math.pi: angle_error += 2 * math.pi
88
89     msg = Twist()
90
91     # 3. P-Controller
92     if distance > 0.1:
93         msg.linear.x = min(2.0, self.Kp_linear * distance)
94         msg.angular.z = self.Kp_angular * angle_error
95     else:
96         self.current_wp_index += 1 # Eat breadcrumb
97         # Progress log every 20 breadcrumbs - helps catch "stuck on
98         # waypoint 0 forever" bugs caused by oscillating gains.
99         if self.current_wp_index % 20 == 0:
100             self.get_logger().info(
101                 f"Progress: breadcrumb "
102                 f"{self.current_wp_index}/{len(self.waypoints)}"
103             )
104
105     self.pub_cmd.publish(msg)
106
107 def stop_robot(self):
108     msg = Twist(); self.pub_cmd.publish(msg)
109     self.get_logger().info("Mission Complete. Press Ctrl+C to exit.")
110
111 def main(args=None):
112     rclpy.init(args=args)
113     node = Autosteer()
114     try:
115         rclpy.spin(node)
116     except KeyboardInterrupt:
117         pass
118     finally:
119         # SAFETY: publish zero velocity on shutdown so the turtle
120         # doesn't coast with the last command. Reuse this pattern
121         # in every ROS 2 node you write.
122         stop_msg = Twist()
123         node.pub_cmd.publish(stop_msg)
124         node.destroy_node()
125         rclpy.shutdown()
126
127 if __name__ == '__main__':

```

Listing 1: autosteer.py (Base Logic)

Step-by-Step Code Walkthrough

Let's break down the key parts of the control loop in plain language.

1. Selecting the Current Waypoint

```
target_x, target_y = self.waypoints[self.current_wp_index]
```

The robot only focuses on one waypoint at a time, the next “breadcrumb” in the list.

Idea: It is not trying to solve the whole path, just the next small step.

2. Calculating Position Error

```
dx = target_x - self.pose.x
dy = target_y - self.pose.y
```

This computes how far off the robot is in the X and Y directions.

Interpretation:

- dx = left/right error
- dy = forward/backward error

3. Distance to the Waypoint

```
distance = math.sqrt(dx**2 + dy**2)
```

This uses the **Pythagorean theorem**:

$$distance = \sqrt{dx^2 + dy^2}$$

Meaning: “How far am I from the current waypoint?”

4. Direction to the Waypoint

```
angle_to_goal = math.atan2(dy, dx)
```

The function `atan2` calculates the angle from the robot to the waypoint.

Meaning: “Which direction should I be pointing?”

5. Heading Error

```
angle_error = angle_to_goal - self.pose.theta
```

This compares:

- where the robot is currently facing
- where it should be facing

Meaning: “How far off is my steering direction?”

6. Waypoint Capture (Eating the Breadcrumb)

```
if distance > 0.1:
    ...
else:
    self.current_wp_index += 1
```

If the robot is within **0.1 units** of the waypoint, it is considered reached.

Interpretation:

- Outside 0.1 → keep driving
- Inside 0.1 → move to next waypoint

Key Idea: Each waypoint has an invisible “capture radius.”

7. Steering Control (P-Controller)

```
msg.angular.z = self.Kp_angular * angle_error
```

This is the proportional control rule:

$$\text{Turn Rate} = K_p \times \text{Error}$$

- Large error → turn quickly
- Small error → turn gently

8. Forward Speed Control

```
msg.linear.x = min(2.0, self.Kp_linear * distance)
```

The robot moves faster when far away and slows down as it approaches the waypoint.

Important:

- Distance large → fast movement
- Distance small → slow movement (smooth stop)

How the P-Controller Works

Look closely at the `control_loop` function. This is the brain of the robot.

1. Calculate Error (The Difference):

```
angle_error = angle_to_goal - self.pose.theta
```

The robot asks: “I want to face 90°, but I am facing 80°. My error is +10°.”

2. Normalize (The Smart Fix):

```
if angle_error > math.pi: angle_error -= 2 * math.pi
```

If the error is 350° (Left), the robot realizes it’s actually just −10° (Right). This prevents it from doing a full donut to correct a small mistake.

3. The P-Controller (The Action):

```
msg.angular.z = self.Kp_angular * angle_error
```

This is the P-Controller formula: $\text{Output} = \text{Gain} \times \text{Error}$.

- If error is **Large** (90°) → Turn **Fast**.
- If error is **Small** (1°) → Turn **Slow**.
- If error is **Zero** → Stop Turning.

Part 3: Registration & Build

The code is in the folder, but ROS doesn't know it is an executable program yet. We must register it in the "Phonebook" (`setup.py`) and then build the workspace.

1. Open `setup.py`:

```
1 code /workspaces/agtech_ros2/src/lab7_autosteer/setup.py
2
```

2. Add the Entry Point: Find the `console_scripts` array and make it look like this:

```
1     entry_points={
2         'console_scripts': [
3             'autosteer = lab7_autosteer.autosteer:main',
4         ],
5     },
6
```

3. Build and Source:

```
1 cd /workspaces/agtech_ros2
2 colcon build --symlink-install
3 source install/setup.bash
4
```

Critical Warning

If you skip the `colcon build` step, you will see an error saying Package 'lab7_autosteer' not found. You must rebuild every time you add a new package or change `setup.py`.

Part 4: The Test Drive

Before you attempt the challenge, let's verify the base code works. You should see the robot perform a perfect "J-Turn" (Straight → Left Turn).

1. Terminal 1: The Simulator

```
1 ros2 run turtlesim turtlesim_node
2
```

Note: If the window doesn't open on a Linux machine, run `xhost +local:root` in your host terminal, and re-export display.

2. Terminal 2: Your Autosteer Node (Don't forget to source!)

```
1 source install/setup.bash
2 ros2 run lab7_autosteer autosteer
3
```

Expected Outcome

The turtle should drive East for 8 metres, then smoothly curve 90 degrees North, and stop facing Up (+Y). The node prints "Mission Complete" when done; press Ctrl+C to exit.

- If it spins in place: Your `linear.x` is zero (Tank Turn).
- If it drives straight forever: Your `angular.z` logic is broken.
- If the progress log stops at "breadcrumb 0/109": your `Kp.angular` is too high, the turtle is oscillating around the first breadcrumb and never getting within 0.1 m. Lower the gain and rebuild.

Part 5: Challenge Completing the U-Turn

Run the code as it is now. You will see something impressive: the robot drives straight, and then smoothly curves 90 degrees to the North (a “J-Turn”). It demonstrates that the P-Controller is actively steering, not just guessing!

Your Task: Complete the U-Turn and bring the tractor home.

1. **Analyze Segment 2:** Look at how we generated the arc. We used the parametric circle equation:

$$x = center_x + radius \times \cos(\theta)$$

$$y = center_y + radius \times \sin(\theta)$$

We iterated θ from -1.57 ($-\pi/2$, Bottom of circle) to 0 (Middle of circle).

2. **Implement Segment 3 (The Second 90 Degrees):** Add a new loop to continue the arc.

- Start Angle: 0
- End Angle: $+\pi/2$ (Top of circle)
- Result: The robot should end up at $(9.0, 2.0)$ facing West.

3. **Implement Segment 4 (The Return Trip):** Use Linear Interpolation (copy Segment 1 logic) to drive from $(9.0, 2.0)$ back to $(1.0, 2.0)$.

AgTech Insight: Why not just spin?

Notice that we force the robot to keep moving while turning ($radius = 0.5$). Real tractors cannot spin in place; they must drive an arc to turn. This path plan is physically possible for a heavy machine; a simple “Rotate 180” command is not.

Industry Context: The “Nav2” Stack

You might be wondering: *“Do real roboticists write P-control loops from scratch?”* Sometimes, but usually they use the **ROS 2 Navigation Stack (Nav2)**.

- **What it is:** An industrial-grade library that handles path planning (A*), obstacle avoidance (Costmaps), and recovery behaviours (Backing up) automatically.
- **Why we aren’t using it yet:** Nav2 is a “Black Box.” If you don’t understand the underlying P-control math (what you just wrote), you will never be able to tune Nav2 when your robot inevitably gets stuck in the mud.
- **The AgTech Goal:** By building this Autosteer node yourself, you are learning the physics necessary to master the pro-level tools later.

Part 6: Controller Tuning and Behaviour

Now that your robot can follow a path, we will explore how its behaviour changes under different settings. Real robotic systems are rarely tuned once. They are adjusted and tested repeatedly.

Experiment 1: Gain Tuning (Kp)

The parameter `Kp_angular` controls how aggressively the robot turns.

- Try values such as: 1.0, 2.0, 4.0, 8.0

Observe the behaviour:

- Low gain: the robot turns slowly and drifts off path
- Medium gain: the robot follows the path smoothly
- High gain: the robot oscillates or jitters

Question: What value gave you the best performance? Why?

Experiment 2: Path Resolution

The parameter `resolution` determines how many waypoints are generated.

- Try values such as: 0.05, 0.1, 0.5

Observe:

- Small resolution → smoother path (more waypoints)
- Large resolution → rough, blocky motion

Question: Why does increasing resolution improve path accuracy?

Challenge: Lookahead Steering

Currently, the robot targets the **current waypoint**.

A more advanced approach is to aim slightly ahead:

```
lookahead_index = min(
    self.current_wp_index + N,
    len(self.waypoints) - 1
)
```

Use this index when selecting the target waypoint instead of `self.current_wp_index`.

Try: $N = 3, 5, 10$

Observe:

- Motion becomes smoother
- Robot anticipates turns instead of reacting late

Concept: Instead of chasing the breadcrumb directly under it, the robot chases a point slightly ahead.

Question: How does lookahead change the robot's turning behaviour?

Deliverables

Submit a single PDF containing:

1. **Path Execution** A screenshot showing the full “paperclip” trajectory in turtlesim (Straight → U-turn → return row).
2. **Code Snippets** Include your implementation for:
 - Segment 3 (second 90-degree turn)
 - Segment 4 (return path)
3. **Gain Tuning Results** Report the value of `Kp_angular` that worked best.
Include a short explanation:
 - What happened when it was too low?
 - What happened when it was too high?
4. **Resolution Experiment** Describe how changing `resolution` affected:
 - Path smoothness
 - Robot behaviour
5. **Concept Question** Explain why this line is important:

```
msg.linear.x = min(2.0, self.Kp_linear * distance)
```

- What happens to the robot's speed as it approaches a waypoint?
 - What would happen if the `min(2.0, ...)` was removed?
6. **Challenge (Optional)** If you implemented lookahead steering:
- What value of N did you use?
 - How did the behaviour change compared to the original controller?